

Objecten & Arrays

INFO

Lesinhoud

Objecten & Arrays

Doelstellingen

- de basisprincipes van Javascript-programmatie (variabelen en constanten, controlestructuren, functies, arrays, objecten, DOM en DOM manipulatie, events en event handling) toepassen binnen een project;

▼ Inhoud

- [Intro](#)
- [Objecten](#)
 - [Een object aanmaken](#)
 - [Een object wijzigen](#)
 - [Waarden van een object uitlezen](#)
 - [Een eigenschap verwijderen](#)
- [Arrays](#)
 - [Een array aanmaken](#)
 - [Waarden uit een array lezen](#)
 - [Een element in een array wijzigen](#)
 - [Het aantal elementen opvragen](#)
 - [Elementen toevoegen en verwijderen](#)

- [Arrays en Strings manipuleren](#)
 - [Conclusie: Objecten vs. Arrays](#)
 - [Wanneer gebruik je een Object?](#)
 - [Wanneer gebruik je een Array?](#)
 - [De gouden combinatie: Objecten in Arrays](#)
-

Intro

Een website bevat vaak niet enkel statische content. In de realiteit komen er vaak heel wat verschillende soorten gegevens aan te pas om delen of secties van een website te kunnen tonen. Denk bijvoorbeeld aan een reeks producten die te koop zijn op een website. Indien deze lijst van producten statisch geprogrammeerd zou zijn, wil dat zeggen dat telkens wanneer een product gewijzigd, toegevoegd of verwijderd moet worden, een developer deze wijzigingen handmatig moet programmeren. Dit is niet efficiënt en wordt in de praktijk anders aangepakt.

In de praktijk worden zulke gegevens meestal opgeslagen in een database. Een webpagina kan een verzoek sturen om de nodige gegevens op te halen uit de database om deze gegevens vervolgens te tonen op de website.

In dit hoofdstuk zullen we de gegevens declareren in ons script in plaats van effectief de gegevens op te vragen uit een database. Gegevens komen ook zeker niet altijd uit een database. Je zal dus ook vaak zelf gegevens zoals objecten en arrays declareren in je code.

Denk eraan dat gegevens die we handmatig toekennen aan variabelen later ook vanuit een database zouden kunnen komen.

Objecten

Bekijk de [MDN Web Docs](#)

Een **object** is een **verzameling van gerelateerde gegevens en functionaliteit**.

Het is een manier om gegevens die bij elkaar horen en die meestal samen één geheel vormen te modelleren/structureren. Je kan een object dus zien als een "container" of "kast" voor **eigenschappen (properties)** en **methoden (methods)** die samen gegroepeerd horen te blijven.

▼ METAFOOR

Een object is als een kast waar je zelf labels kan toekennen aan elk schap in de kast.

In JavaScript hebben we geen kast, maar wel de accolades **{}**, hiermee maak je een object aan. Of in woorden van de metafoor, zetten we een kast waar we gegevens in gaan opslaan klaar.

```
const persoon = {};
```

js

In JavaScript hebben we geen echte labels om op een kast te plakken, maar we hebben eigenschappen (Engels: properties). De waarden die gekoppeld worden aan de eigenschappen, zijn de gegevens die we in het schap van de kast leggen.

```
const persoon = {  
  label1: "waarde 1",  
  label2: "waarde 2",  
};
```

js

Aangezien we zelf kunnen kiezen wat we op het label plaatsen (hoe we de eigenschap noemen), kunnen we dit gebruiken om allerlei gegevens bij te houden.

Stel dat we de informatie over een persoon willen bijhouden, laten we hem Kevin noemen. Kevin is 180cm groot, heeft blonde haren en weegt 75kg. Dit zou resulteren in een kast met 3 schappen, waarbij 1 schap het label lengte heeft, 1 schap het label haarkleur en 1 schap het label gewicht. Met in elk schap respectievelijk de informatie 180, blond en 75.

```
const persoon = {  
  lengte: 180,  
  haarkleur: "blond",  
  gewicht: 75,  
};
```

js

We zien "de kast", ons object, `{}`. Met daarin 3 "schappen" (**properties**) met elk hun eigen label: lengte, haarkleur en gewicht. In elk schap zit de bijhorende waarde: 180, "blond" en 75.

Een object bestaat uit **key-value pairs** (Nederlands: sleutel-waarde-paren), waarbij de **key** een unieke naam is en waarbij de **value** de gegevens voorstellen.

Er is een onderscheid tussen gewone properties en methods.

- **Eigenschappen (properties):** Een key waaraan je een **number**, **string**, **boolean**, **array**, **object**, ... toekent.
- **Methoden (Methods):** Een key waaraan je een functie toekent.

Functies

Functies zijn blokken code die je op een later tijdstip kan uitvoeren, dit wordt in een ander hoofdstuk in meer detail bekeken.

Een object aanmaken

Een object wordt geïnitieerd d.m.v. 2 kromme haakjes (curly braces)

`{}`.

js

```
const product = {}; // Dit is een leeg object. Hieraan  
kunnen later in de code nog eigenschappen (properties)  
worden toegevoegd.
```

Als je het object al meteen wil opvullen met gegevens tijdens het initialiseren van de variabele kan dat door de key-value pairs tussen de haken te plaatsen:

js

```
const product = {  
  naam: "Hogeschool PXL sweater",  
  prijs: 50,  
  materiaal: "Katoen",  
}; // Het object heeft nu drie eigenschappen  
(properties): naam, prijs en materiaal
```

▼ Oefening 1

Maak een object `boek` met de eigenschappen:

- `titel: "JavaScript Basis"`
- `auteur: "PXL"`
- `paginas: 200`

Toon het object in de console.

▼ Oplossing

js

```
const boek = {  
  titel: "JavaScript Basis",  
  auteur: "PXL",  
  paginas: 200,  
};  
console.log(boek);
```

▼ Oefening 2

Maak een object `student` met volgende structuur:

- `voornaam: "An"`
- `achternaam: "Peeters"`
- `adres: { straat: "Elfde-Liniestraat", stad: "Hasselt" }`

Toon `student` in de console. Bekijk de waarde in de variabele ook eens in je `DevTools` door een breakpoint te plaatsen aan het einde van je code.

▼ Oplossing

```
const student = {  
  voornaam: "An",  
  achternaam: "Peeters",  
  adres: {  
    straat: "Elfde-Liniestraat",  
    stad: "Hasselt",  
  },  
};  
console.log(student);
```

js

Een object wijzigen

Een object kan ook nieuwe eigenschappen (properties) toegewezen krijgen later in de code. Dit doe je door eerst de variabele waar het object in zit op te roepen. Vervolgens kun je dan via een `.` gevolgd door de naam van een nieuwe of bestaande eigenschap (property) een eigenschap toevoegen of veranderen.

```
const product = {  
  naam: "Hogeschool PXL sweater",  
  prijs: 50,
```

js

```

    materiaal: "Katoen",
  };

  // De eigenschap "kleur" bestond in dit object nog niet.
  Ze wordt simpelweg toegevoegd nieuwe extra
  property/eigenschap van het product-object
  product.kleur = "zwart";

  // De eigenschap "prijs" bestond in dit object al. De
  vorige waarde wordt dus overschreven met de nieuwe waarde
  product.prijs = 60;

```

De waarde van de variabele "product" na het uitvoeren van bovenstaande code:

```

{
  naam: "Hogeschool PXL sweater",
  prijs: 60,
  materiaal: "Katoen",
  kleur: "zwart"
}

```

js

Naast de `.`-notatie (**dot notation**) kunnen we de waarde van een eigenschap van een object ook met de `[]`-notatie (**bracket notation**) opvragen of wijzigen:

```

const product = {
  naam: "Hogeschool PXL sweater",
  prijs: 50,
  materiaal: "Katoen",
};

// De eigenschap (`property`) 'kleur' bestond nog niet.
Het object heeft vanaf nu dus 4 eigenschappen i.p.v. 3.
product["kleur"] = "zwart";

```

js

```
// De eigenschap (`property`) 'prijs' bestond in dit  
object al. De vorige waarde wordt dus overschreven met de  
nieuwe waarde.  
product["prijs"] = 60;
```

▼ Oefening

Gegeven:

```
const gebruiker = {  
  naam: "Youssef",  
  rol: "viewer",  
};
```

js

- Verander de waarde van `rol` naar `"editor"` met bracket-notation.
- Voeg een eigenschap `ingelogd` toe met waarde `true` met de dot-notation.

▼ Oplossing

```
const gebruiker = {  
  naam: "Youssef",  
  rol: "viewer",  
};  
gebruiker["rol"] = "editor";  
gebruiker.ingelogd = true;
```

js

Waarden van een object uitlezen

De waarden van een object uitlezen gaat heel gemakkelijk. Hiervoor gebruiken we dezelfde technieken die we gebruikt hebben om eigenschappen toe te voegen of te wijzigen.

We kunnen hier dus opnieuw kiezen om de `.`-notatie te gebruiken of de `[]`-notatie.

```
const product = {
  naam: "Hogeschool PXL sweater",
  prijs: 50,
  materiaal: "Katoen",
};

console.log("Het product kost: " + product.prijs); //
optie 1 om de waarde van een eigenschap uit te lezen.
console.log("Het product kost: " + product["prijs"]); //
optie 2 om de waarde van een eigenschap uit te lezen.

const productNaamLengte = product.naam.length; // Een
voorbeeld van hoe we het aantal karakters in de naam
kunnen opvragen.
const tienvoud = product.prijs * 10; // Een voorbeeld hoe
een waarde van een eigenschap gebruikt kan worden in een
berekening.
```

▼ Oefening 1

Gegeven een object met een eigenschapsnaam met een koppelteken:

```
const profiel = {
  gebruikersnaam: "pxlstudent",
  "favoriete-drank": "koffie",
};
```

Lees de waarde van `favoriete-drank` uit met **bracket notation** en stop die in de variabele `drank`.

▼ Oplossing

```
const profiel = {
  gebruikersnaam: "pxlstudent",
```

```
"favoriete-drank": "koffie",
};
const drank = profiel["favoriete-drank"];
```

▼ **Oefening 2: lees onderstaande tip zeker volledig door en stel gerichte vragen wanneer je iets niet begrijpt!**

Toon de gegevens van het student-object in een paragraaf op de pagina. Hierbij krijg je een beetje verder wat hulp, geen zorgen.

Zorg ervoor dat de gegevens van onderstaande student **in een paragraaf** op de pagina worden weergegeven. De gegevens moeten in het volgende formaat in de paragraaf verschijnen:

An Peeters woont in de Elfde-Liniestraat te Hasselt .

Als je twijfelt, kijk dan zeker nog eens naar de tip die werd meegegeven bij deze oefening of stel gerust vragen aan je lector.

Gegeven:

```
const student = {
  voornaam: "An",
  achternaam: "Peeters",
  adres: {
    straat: "Elfde-Liniestraat",
    stad: "Hasselt",
  },
};
```

js

/*

Stap 1: Maak een nieuwe paragraaf aan en sla deze op in een variabele (Tip: createElement())

Stap 2: Bouw de string string op die alle gegevens bevat van het student-object in het gevraagde formaat

Stap 3: Stel de string in als de tekst/content van de paragraaf die je via createElement() hebt aangemaakt. (Tip: .textContent of .innerText)

Stap 4: Voeg de paragraaf aan de body van je pagina toe. (Tip: appendChild())

*/

▼ Oplossing

```
const student = {  
  voornaam: "An",  
  achternaam: "Peeters",  
  adres: {  
    straat: "Elfde-Liniestraat",  
    stad: "Hasselt",  
  },  
};  
  
// maakt een paragraaf-element aan en slaat deze op in  
// een variabele  
const pStudentInfo = document.createElement("p");  
  
// Haal de waardes uit het student-object en concateneer  
// ze tot één lange string  
let studentInfo = `${student.voornaam} +  
${student.achternaam} woont in de  
${student.adres.straat} te ${student.adres.stad}`;  
  
// Stel de studentInfo value, een string, nu in als de  
// content of inhoud van de paragraaf die je net hebt  
// aangemaakt  
pStudentInfo.innerText = studentInfo;  
  
// appendChild() zal het meegegeven element als child-  
// element toevoegen binnen het element waar we de functie  
// op oproepen.  
// In dit geval zal de paragraaf dus aan de body van je  
// pagina worden toegevoegd.  
document.body.appendChild(pStudentInfo);
```

js

Tip voor oefening 2

Gebruik de [createElement](#)- en [appendChild](#)-methodes om nieuwe elementen te creëren en toe te voegen aan de pagina.

```
// Onderstaand lijn code maakt hier een nieuw div-element aan  
// Verander "div" naar een andere tag als je een ander
```

js

```
element wilt maken
const container = document.createElement("div");

// Voeg het element dat is opgeslaan in de container-
variabele toe aan de body van je pagina d.m.v de appendChild-
methode
document.body.appendChild(container);

const title = document.createElement("h2");
title.textContent = "Tekst van de titel"; // .innerText kan
hier ook gebruikt worden i.p.v. .textContent

// Voeg het h2-element toe aan de div in de container-
variabele
container.appendChild(title);
```

Een eigenschap verwijderen

Bekijk de [MDN Web Docs](#)

Om een eigenschap te verwijderen gebruiken we het `delete-keyword`, gevolgd door de eigenschap die we willen verwijderen.

```
delete product.naam;
```

js

Arrays

Bekijk de [MDN Web Docs](#)

Waar een object een verzameling van gerelateerde eigenschappen is, is een array `een geordende lijst van waarden`. Denk aan een boodschappenlijstje, een lijst van studenten, of een reeks metingen. De volgorde van de items in een array is belangrijk en blijft behouden.

Elk item in een array heeft een index: een numerieke positie die aangeeft waar in de lijst het item zich bevindt. Belangrijk om te weten is dat JavaScript, net als de meeste programmeertalen, nul-geïndexeerd is. Dit betekent dat het tellen van de posities begint bij 0. Het eerste item heeft index 0, het tweede item index 1, enzovoort.

▼ METAFOOR

Een array is als een kast waar je automatisch labels met opeenvolgende nummers worden toegevoegd aan elk schap in de kast. In JavaScript hebben we geen kast, maar wel de vierkante haken [], hiermee maak je een array aan. Of in woorden van de metafoor, zetten we een kast klaar.

```
const studenten = [];
```

js

In tegenstelling tot objecten, waarbij we zelf labels op de schappen plakken, wordt hier automatisch een label met een cijfer aan elk schap toegekend.

```
const studenten = ["Kwik", "Kwek", "kwak"];
```

js

Nu zou je iets moeten opvallen; Bij objecten zagen we de labels (de **properties**), maar bij arrays niet. Dit komt door de automatische labels, elk eerste item zal altijd het label 0 hebben, het tweede item zal altijd het label 1 hebben, ...

We zien in bovenstaand voorbeeld een "kast" [] met 3 schappen:

- Schap 1 (label 0) met als waarde "Kwik"
- Schap 2 (label 1) met als waarde "Kwek"
- Schap 3 (label 2) met als waarde "Kwak"

Met de juiste termen zouden we dit verwoorden als: We zien in bovenstaand voorbeeld een array [], met daarin 3 elementen:

- Het eerste element (index 0) bevat de waarde "Kwik"
- Het tweede element (index 1) bevat de waarde "Kwek"
- Het derde element (index 2) bevat de waarde "Kwak"

Een array aanmaken

Een array wordt aangemaakt met vierkante haken `[]`.

```
const legeLijst = []; // Een lege array, klaar om gevuld te worden.

const fruitsoorten = ["Appel", "Banaan", "Kers"]; // Een array met een lijst van strings (tekst).

const scores = [15, 12, 18, 9]; // Een array met een lijst van getallen.

const gemengdeLijst = ["Tekst", 100, true, { naam: "PXL" }]; // Een array kan ook verschillende datatypes bevatten.
```

js

Waarden uit een array lezen

Om een specifieke waarde uit een array te halen, gebruik je de naam van de array gevolgd door de index van het gewenste item tussen vierkante haken `[]`.

```
<h1 id="titel"></h1>
```

html

```
const fruitsoorten = ["Appel", "Banaan", "Kers"];

console.log(fruitsoorten[0]); // Geeft "Appel" weer
console.log(fruitsoorten[2]); // Geeft "Kers" weer

// Waarde tonen in een HTML-element op de pagina.
const titelElement = document.querySelector("#titel");
titelElement.innerText = `Mijn favoriete fruit is`
```

js

```
${fruitsoorten[1]}`; // Titel wordt "Mijn favoriete fruit  
is Banaan"
```

Mijn favoriete fruit is Banaan

Een element in een array wijzigen

Op dezelfde manier kan je een bestaand element in een array een nieuwe waarde geven.

```
const scores = [15, 12, 18, 9];  
scores[3] = 10; // Het vierde element (index 3) was 9, en  
wordt nu 10.  
  
console.log(scores); // Geeft [15, 12, 18, 10] weer
```

js

```
<div class="output"></div>  
<div class="output-formatted"></div>
```

html

```
const scores = [15, 12, 18, 9];  
scores[3] = 10; // Het vierde element (index 3) was 9, en  
wordt nu 10.  
  
const outputDiv = document.querySelector(".output");  
outputDiv.innerText = `De aangepaste scores zijn:  
${scores}`;  
  
const outputFormattedDiv =  
document.querySelector(".output-formatted");  
outputFormattedDiv.innerText = `De aangepaste scores  
zijn: ${scores.join(" - ")}`
```

js

De aangepaste scores zijn: 15,12,18,10
De aangepaste scores zijn: 15 - 12 - 18 - 10

Oefening

Pas in de bovenstaande code de waarde van het tweede element (index 1) aan naar 13. Pas in bovenstaande code het tonen van de aangepaste scores aan, zodat de scores met een komma en spatie ertussen getoond worden.

```
15, 13, 18, 10
```

Het aantal elementen opvragen

Bekijk de [MDN Web Docs](#)

De `.length`-eigenschap geeft je het totale aantal items in de array. Dit is handig, want de index van het laatste element is altijd `length - 1`. Bij het itereren later in deze cursus gaat dit handig van pas komen.

```
const studenten = ["An", "Bart", "Youssef", "Davina"];
console.log(studenten.length); // Geeft 4 weer

const laatsteStudent = studenten[studenten.length - 1];
// De index van het laatste element is 3.
console.log(laatsteStudent); // Geeft "Davina" weer
```

js

Elementen toevoegen en verwijderen

JavaScript biedt standaard verschillende methoden om arrays aan te passen.

Gegevens aan het einde toevoegen of verwijderen: push() en pop()

- `push()` : Voegt één of meerdere elementen toe aan het einde van de array.
- `pop()` : Verwijdert het laatste element van de array en geeft dit element terug.

```
const taken = ["Afwassen", "Stofzuigen"];

taken.push("De les scripting nakijken");
console.log(taken); // ["Afwassen", "Stofzuigen", "De les scripting nakijken"]

const verwijderdeTaak = taken.pop();
console.log(verwijderdeTaak); // "De les scripting nakijken"
console.log(taken); // ["Afwassen", "Stofzuigen"]
```

js

▼ Oefening 1

Voeg aan onderstaande array het "ragdoll" kattenras toe. Verwijder ze daarna weer meteen uit de array.

```
let catBreeds = ["Persian", "Maine Coon", "Siamese"];
```

js

▼ Oplossing

```
let catBreeds = ["Persian", "Maine Coon", "Siamese"];
catBreeds.push("Ragdoll");
catBreeds.pop();
```

js

▼ Oefening 2

Maak een nieuwe array "travelDestinations" aan. Vul deze met drie bestemming-objecten. Elk object bezit de eigenschappen 'city' en 'country'.

- Wijzig het eerste object naar `city: "Hasselt"` en `country: "Belgium"`
- Verwijder het laatste object uit de array
- Voeg Sydney, Australia aan de array toe

▼ Oplossing

```
let travelDestinations = [  
  { city: "Tokyo", country: "Japan" },  
  { city: "New York", country: "USA" },  
  { city: "Parijs", country: "Frankrijk" },  
];  
  
let removedDestination = travelDestinations.pop();  
  
let newDestination = { city: "Sydney", country:  
  "Australia" };  
travelDestinations.push(newDestination);  
// Het volgende mag ook:  
// travelDestinations.push({ city: "Sydney", country:  
  "Australia" });
```

Gegevens aan het begin toevoegen of verwijderen: unshift() en shift()

- `unshift()` : Voegt één of meerdere elementen toe aan het begin van de array.
- `shift()` : Verwijdert het eerste element van de array en geeft dit element terug.

js

```
const wachtrij = ["Persoon 2", "Persoon 3"];

wachtrij.unshift("Persoon 1");
console.log(wachtrij); // ["Persoon 1", "Persoon 2",
                        "Persoon 3"]

const volgendeInRij = wachtrij.shift();
console.log(volgendeInRij); // "Persoon 1"
console.log(wachtrij); // ["Persoon 2", "Persoon 3"]
```

▼ Oefening 1

Verwijder uit onderstaande array eerst het eerste item en voeg daarna aan het begin van de array het element "Peer" toe.

Valideer of "Appel" echt uit de array is verwijderd door een breakpoint te plaatsen in je `DevTools` onder `bronnen`, `sources` of `debugger`.

js

```
let fruit = ["Appel", "Banaan"];
```

▼ Oplossing

js

```
const fruit = ["Appel", "Banaan"];

// verwijder het eerste element
fruit.shift();

// voeg "Peer" aan het begin toe
fruit.unshift("Peer");

// Plaats hier een breakpoint om de waarde van fruit te
controleren
```

▼ Oefening 2

Je hebt een array van objecten die fruit vertegenwoordigen. Voeg een nieuw object toe aan het begin van de array met als naam "mango" en waarbij kleur gelijk is aan de kleur van het laatste fruit in de array.

Valideer het resultaat van je code door een breakpoint te plaatsen in je

`DevTools` onder `bronnen`, `sources` of `debugger`.

```
const fruitMand = [  
  { naam: "Appel", kleur: "rood" },  
  { naam: "Banaan", kleur: "geel" },  
];
```

js

▼ Oplossing

```
const fruitMand = [  
  { naam: "Appel", kleur: "rood" },  
  { naam: "Banaan", kleur: "geel" },  
];  
  
// bepaal de kleur van het laatste element  
const laatsteKleur = fruitMand[fruitMand.length -  
1].kleur;  
  
// voeg vooraan Mango toe met dezelfde kleur  
fruitMand.unshift({ naam: "Mango", kleur: laatsteKleur  
});  
  
// Plaats hier een breakpoint om de waarde van fruitMand  
te controleren
```

js

De veelzijdige splice-methode

Oefening alleen of per twee:

Bekijk de [MDN Web Docs](#) en probeer zelfstandig uit te zoeken wat de

`splice()`-methode allemaal kan en hoe deze gebruikt kan worden.

Probeer daarna de oefeningen op te lossen.

▼ Oefening 1

Gegeven:

```
const dagen = ["ma", "di", "wo", "do", "vr"];
```

js

- Verwijder het element `"wo"` met `splice` en bewaar het verwijderde element in de variabele `verwijderd`.
- Toon zowel de aangepaste array als `verwijderd` in de console.

▼ Oplossing

```
const dagen = ["ma", "di", "wo", "do", "vr"];
const verwijderd = dagen.splice(2, 1); // vanaf index 2,
verwijder 1 element
console.log(dagen); // ["ma", "di", "do", "vr"]
console.log(verwijderd); // ["wo"]
```

js

▼ Oefening 2

Gegeven:

```
const kleuren = ["rood", "groen", "blauw"];
```

js

- Vervang `"groen"` door `"geel"` met `splice`.
- Voeg `"paars"` tussen `"geel"` en `"blauw"` in met `splice`.

Valideer het resultaat van je code door een breakpoint te plaatsen in je `DevTools` onder `bronnen`, `sources` of `debugger`.

▼ Oplossing

```
const kleuren = ["rood", "groen", "blauw"];
kleuren.splice(1, 1, "geel"); // vervang vanaf index 1,
```

js

```
1 element door "geel"  
kleuren.splice(2, 0, "paars"); // voeg in op index 2,  
verwijder 0  
// Resultaat: ["rood", "geel", "paars", "blauw"]
```

Arrays en Strings manipuleren

De `join()` -methode: Een Array omzetten naar een String

Bekijk de [MDN Web Docs](#)

De `join()` -methode plakt alle elementen van een array aan elkaar tot één string. Je kan een "scheidingsteken" (separator) meegeven om tussen de elementen te plaatsen.

```
const woorden = ["Hallo", "wereld", "!"];  
  
const zinZonderSpaties = woorden.join(""); //  
"Hallowereld!"  
const zinMetSpaties = woorden.join(" "); // "Hallo wereld  
!"  
const zinMetKoppelteken = woorden.join("-"); // "Hallo-  
wereld-!"
```

js

▼ Oefening 1

Gegeven:

```
const datumOnderdelen = ["08", "11", "2025"];
```

js

Zet de array om naar de string `"08/11/2025"` met `join` en stop die in de variabele `datum`.

Valideer het resultaat van je code door een breakpoint te plaatsen in je

`DevTools` onder `bronnen`, `sources` of `debugger`.

▼ Oplossing

```
const datumOnderdelen = ["08", "11", "2025"];  
const datum = datumOnderdelen.join("/");
```

js

▼ Oefening 2

Gegeven:

```
const onderdelen = ["app", "js", "map"];
```

js

Maak de string `"app.js.map"` met `join` en bewaar die in `bestandsnaam`.
Valideer het resultaat van je code door een breakpoint te plaatsen in je `DevTools` onder `bronnen`, `sources` of `debugger`.

▼ Oplossing

```
const onderdelen = ["app", "js", "map"];  
const bestandsnaam = onderdelen.join(".");
```

js

De `split()` -methode: Een String omzetten naar een Array

Bekijk de [MDN Web Docs](#)

De `split()` -methode is het omgekeerde van `join()`. Het is een methode op een string, niet op een array. Het splitst een string op in een array van substrings, gebaseerd op een scheidingsteken dat je meegeeft.

```
const zin = "PXL is een hogeschool";  
const woordenArray = zin.split(" "); // Splits de string  
op elke spatie. Je kan hier ook een andere string om op
```

js

te splitsen meegeven.

```
console.log(woordenArray); // ["PXL", "is", "een",  
"hogeschool"]
```

▼ Oefening 1

Gegeven:

```
const tekst = "Jan;Piet;Klaas";
```

js

- Splits `tekst` op `;` naar een array `namen`.
- Toon het **tweede** element uit `namen` in de console.

▼ Oplossing

```
const tekst = "Jan;Piet;Klaas";  
const namen = tekst.split(";");  
console.log(namen[1]); // Piet
```

js

▼ Oefening 2

Gegeven:

```
const invoer = "BE-0123456789";
```

js

- Splits `invoer` op `-`.
- Bewaar het linkerdeel in `landCode` en het rechterdeel in `nummer`.

Valideer het resultaat van je code door een breakpoint te plaatsen in je `DevTools` onder `bronnen`, `sources` of `debugger`.

▼ Oplossing


```
const invoer = "BE-0123456789";  
const delen = invoer.split("-");  
const landCode = delen[0];  
const nummer = delen[1];
```

js

Conclusie: Objecten vs. Arrays

Zowel objecten als arrays zijn fundamentele bouwstenen in JavaScript voor het beheren van data. Hoewel ze beiden meerdere waarden kunnen bevatten, hebben ze een ander doel en worden ze in verschillende situaties gebruikt.

Wanneer gebruik je een Object?

Gebruik een object als je een enkel, complex geheel wilt voorstellen met gerelateerde eigenschappen. De volgorde van die eigenschappen is niet belangrijk, maar wel de naam (de key) die een beschrijving geeft van de waarde. Objecten zijn perfect om de structuur van iets uit de echte wereld te modelleren.

Denk aan:

- Een student: heeft een voornaam, achternaam, studentnummer en opleiding.
- Een product: heeft een naam, prijs, kleur en materiaal.
- Een configuratie: heeft instellingen zoals thema: 'donker', taal: 'nl' en notificaties: true.

Wanneer gebruik je een Array?

Gebruik een array als je een geordende lijst van (meestal) gelijksoortige items wilt bijhouden. Hier is de volgorde wél belangrijk en spreek je de elementen aan via hun numerieke index (0, 1, 2, ...).

Denk aan:

- Een boodschappenlijst: ["melk", "brood", "eieren"].
- Een lijst van scores voor een examen: [14, 18, 9, 15].
- De gebruikers in een wachtrij.

De gouden combinatie: Objecten in Arrays

In de praktijk worden objecten en arrays heel vaak gecombineerd om complexe datastructuren te maken. De meest voorkomende combinatie is een array van objecten. Dit is de standaardmanier om een lijst van gestructureerde data weer te geven, zoals je die bijvoorbeeld uit een database zou ontvangen.

Stel je voor dat je een lijst van producten wilt tonen op een webshop. Elk product is een object met zijn eigen eigenschappen. Samen vormen ze een array:

```
const producten = [  
  {  
    naam: "Hogeschool PXL sweater",  
    prijs: 60,  
    kleur: "zwart",  
  },  
  {  
    naam: "Hogeschool PXL T-shirt",  
    prijs: 25,  
    kleur: "wit",  
  },  
  {  
    naam: "Hogeschool PXL drinkbus",
```

js

```
    prijs: 15,  
    kleur: "grijs",  
  },  
];  
  
// Nu kun je eenvoudig over de lijst itereren en de  
gegevens laten zien op de pagina.  
producten.forEach((product) => {  
  const p = document.createElement("p");  
  p.innerText = `Het product "${product.naam}" kost €  
${product.prijs}.`;   
  document.body.appendChild(p);  
});
```

Het product "Hogeschool PXL sweater" kost €60.

Het product "Hogeschool PXL T-shirt" kost €25.

Het product "Hogeschool PXL drinkbus" kost €15.

Previous page
[Oefeningen](#)

Next page
[Oefeningen](#)